

Opis procedury testowej

na podstawie pliku app.test.js

Projekt: NBP Currency Analyzer

Zakres: testy automatyczne app.test.js uruchamiane w GitHub Actions po push/PR oraz możliwe do uruchomienia lokalnie

Element	Wartość
Plik testowy	app.test.js
Framework	Jest
Środowisko	JSDOM
Liczba przypadków testowych	18
Testowana aplikacja	NBP Currency Analyzer

Data przygotowania dokumentu: 2026-07-05

1. Cel procedury testowej

Celem procedury testowej jest sprawdzenie poprawności działania głównych modułów JavaScript aplikacji NBP Currency Analyzer oraz częściowa weryfikacja przepływu użytkownika w symulowanym środowisku DOM.

Procedura bazuje bezpośrednio na pliku app.test.js. Testy nie uruchamiają pełnej aplikacji w rzeczywistej przeglądarce. Zamiast tego sprawdzają logikę aplikacji, integrację klas JavaScript oraz wybrane reakcje interfejsu w środowisku JSDOM.

Najważniejsze cele testowania:

- zweryfikowanie działania serwisu odpowiedzialnego za pobieranie i normalizację danych z API NBP;
- sprawdzenie poprawności obliczeń statystycznych i analizy sesji;
- sprawdzenie generowania rozkładu zmian dla par walutowych;
- sprawdzenie integracji klas NBPSERVICE, AnalysisService i CurrencyAnalyzer;
- sprawdzenie generowania tekstu CSV;
- sprawdzenie podstawowego przepływu użytkownika w uproszczonym DOM.

2. Podstawa i charakter testów

Plik app.test.js jest zorganizowany w siedem grup testowych opisanych blokami describe(). Łącznie zawiera 18 przypadków testowych. Testy używają frameworka Jest i deklarują środowisko @jest-environment jsdom.

Cecha procedury	Opis
Rodzaj testów	Testy jednostkowe, testy integracyjne modułów JS, testy obsługi danych oraz uproszczona symulacja przepływu użytkownika.
Poziom testowania	Głównie warstwa JavaScript aplikacji, bez pełnego testu wizualnego w przeglądarce.
API NBP	Mockowane przez global.fetch = jest.fn().
DOM	Tworzony w teście przez document.body.innerHTML, nie jest wczytywany pełny index.html.
Canvas	Mockowany przez HTMLCanvasElement.prototype.getContext.
Testy E2E	Nie. Testy nie są pełnymi testami end-to-end w prawdziwej przeglądarce.

3. Środowisko testowe i sposób uruchamiania

W projekcie podstawowym sposobem wykonania testów jest automatyczne uruchomienie ich w pipeline CI po wrzuceniu zmian do repozytorium GitHub. Lokalna komenda npm test jest tą samą komendą, którą workflow wykonuje wewnątrz zadania CI.

Element	Wartość / opis
Główna procedura	Automatyczne uruchomienie testów przez GitHub Actions po wykonaniu push lub pull request.
Plik workflow CI	.github/workflows/ci.yml
Zdarzenia uruchamiające	push oraz pull_request.
Gałęzie objęte CI	main, release, develop.
Runtime w CI	Node.js 20 na ubuntu-latest.
Framework testowy	Jest
Środowisko DOM	jest-environment-jsdom
Plik testowy	app.test.js
Testowany kod	app.js

Element	Wartość / opis
Komendy wykonywane w CI	npm install, następnie npm test.
Uruchomienie lokalne	Opcjonalne, diagnostyczne: npm install oraz npm test.

Fragment logiki workflow CI:

on: push / pull_request na gałęzi main, release, develop

steps:

- checkout kodu
- setup Node.js 20
- npm install
- npm test

Dodatkowo w projekcie znajduje się workflow release-and-deploy.yml uruchamiany po push na gałąź release. Odpowiada on za utworzenie wydania i wdrożenie, natomiast sama procedura testowa app.test.js jest wykonywana w workflow CI.

4. Ogólny przebieg procedury po wrzuceniu zmian na GitHub

Procedura wykonania testów w repozytorium składa się z następujących kroków:

- programista wykonuje commit i push zmian na jedną z gałęzi: main, release lub develop albo tworzy pull request do jednej z tych gałęzi;
- GitHub Actions automatycznie uruchamia workflow CI Pipeline z pliku .github/workflows/ci.yml;
- workflow pobiera kod z repozytorium przy użyciu actions/checkout;
- workflow konfiguruje środowisko Node.js w wersji 20;
- workflow instaluje zależności projektu przez npm install;
- workflow uruchamia testy z pliku app.test.js przez npm test;
- Jest wykonuje wszystkie 18 przypadków testowych i porównuje wyniki rzeczywiste z oczekiwanymi instrukcjami expect();
- procedura zostaje uznana za zaliczoną tylko wtedy, gdy wszystkie przypadki testowe kończą się statusem passed;
- w przypadku niepowodzenia któregośkolwiek testu workflow kończy się błędem i wskazuje nieudaną grupę lub przypadek testowy w logach GitHub Actions.

Komenda npm test nie jest więc opisana jako jedyny ręczny sposób testowania, lecz jako krok wykonywany automatycznie przez pipeline po opublikowaniu zmian w repozytorium.

5. Grupy przypadków testowych

Grupa testów	Liczba	Cel
NBPService Class Tests	3	Sprawdzenie formatowania dat, obsługi tabeli C oraz obsługi błędu 404 z API.
AnalysisService Session Tests	3	Sprawdzenie klasyfikacji sesji jako wzrostowe, spadkowe i bez zmian oraz przypadków brzegowych.
AnalysisService Statistical Measures Tests	4	Sprawdzenie mediany, dominanty, odchylenia standardowego oraz agregacji statystyk.
AnalysisService Distribution Analysis Tests	2	Sprawdzenie synchronizacji dat, kursów krzyżowych oraz przedziałów histogramu.
CurrencyAnalyzer Integration Tests	2	Sprawdzenie przepływu: pobranie danych -> analiza statystyczna oraz pobranie dwóch walut -> rozkład zmian.
CSVExporter Tests	2	Sprawdzenie generowania tekstu CSV i escapowania cudzysłowów.
End-to-End User Flow Simulation	2	Uproszczona symulacja uruchomienia analizy oraz przełączania widoku wykres/tabela w JSDOM.

6. Szczegółowy opis przypadków testowych

6.1. NBPService Class Tests

ID	Test	Procedura	Oczekiwany rezultat
T-01	formatDate should return YYYY-MM-DD	Utworzenie daty 2022-05-15 i przekazanie jej do service.formatDate().	Zwrócony tekst ma format 2022-05-15.
T-02	fetchRates should parse Table C bid/ask to a single mid value	Zamockowanie odpowiedzi API z effectiveDate, bid = 4.10 i ask = 4.30. Wywołanie fetchRates('C', 'USD', ...).	Pierwsza wartość result[0].value wynosi 4.20, czyli średnia z bid i ask.
T-03	fetchRates should throw NBPServiceError on 404	Zamockowanie odpowiedzi fetch z ok = false i status = 404. Wywołanie fetchRates dla tabeli A i USD.	Zostaje rzucony NBPServiceError, komunikat zawiera 'No data', a status ma wartość 404.

6.2. AnalysisService Session Tests

ID	Test	Procedura	Oczekiwany rezultat
T-04	should correctly classify upward, downward, and flat sessions	Przekazanie kursów 4.50, 4.60, 4.40, 4.40 do analyzeSessions().	up = 1, down = 1, flat = 1, rows.length = 3; pierwszy wiersz ma klasę up, ostatni flat.
T-05	should return empty results if only one session is provided	Przekazanie pojedynczego kursu 4.50 do analyzeSessions().	up = 0 oraz rows.length = 0, ponieważ nie istnieje poprzednia sesja do porównania.
T-06	should handle all rates being equal	Przekazanie trzech identycznych kursów 4.0, 4.0, 4.0.	flat = 2, up = 0, down = 0.

6.3. AnalysisService Statistical Measures Tests

ID	Test	Procedura	Oczekiwany rezultat
T-07	median should correctly calculate for odd and even sets	Wywołanie median([1, 5, 3]) oraz median([1, 2, 3, 4]).	Wyniki: 3 oraz 2.5.
T-08	stddev should calculate mean and population standard deviation	Wywołanie stddev([10, 10, 10]) oraz stddev([2, 4, 4, 4, 5, 5, 7, 9]).	Dla pierwszego zbioru mean = 10 i std = 0. Dla drugiego mean = 5 i std = 2.
T-09	mode should find the most frequent rounded value	Wywołanie mode([1.12344, 1.12341, 5.0, 1.12342]).	Po grupowaniu przez toFixed(4) dominanta ma value = 1.1234 i count = 3.
T-10	analyzeStats should aggregate all measures correctly	Przekazanie kursów o wartościach 10, 20, 30 do analyzeStats().	mean = 20, median = 20, cv > 0, min = 10, max = 30.

6.4. AnalysisService Distribution Analysis Tests

ID	Test	Procedura	Oczekiwany rezultat
T-11	should synchronize dates and calculate correct cross-rates	Przekazanie dwóch serii kursów, z których tylko data 2023-01-02 jest wspólna. Wywołanie analyzeDistribution(..., 'monthly').	changes.length = 0, ponieważ do policzenia zmiany potrzebne są co najmniej dwa okresy.
T-12	should correctly bin percentage changes into 10 intervals	Utworzenie 11 miesięcy danych dla waluty A i odpowiadających im danych dla waluty B. Wywołanie analyzeDistribution(..., 'monthly').	changes.length = 10, bins.length = 10, a suma count we wszystkich binach wynosi 10.

6.5. CurrencyAnalyzer Integration Tests

ID	Test	Procedura	Oczekiwany rezultat
T-13	Full flow: getStatistics should fetch and analyze data	Zamockowanie odpowiedzi API z kursami 4.0 i 5.0. Wywołanie analyzer.getStatistics('A', 'USD', ...).	rates.length = 2, stats.mean = 4.5, stats.max = 5.0.
T-14	Full flow: getDistribution should handle parallel fetching and processing	Zamockowanie dwóch odpowiedzi API: USD 4.0 -> 4.4 oraz EUR 1.0 -> 1.0. Wywołanie getDistribution('A', 'USD', 'EUR', ..., 'monthly').	changes.length = 1, a changes[0].change = 10, czyli wzrost kursu krzyżowego o 10%.

6.6. CSVExporter Tests

ID	Test	Procedura	Oczekiwany rezultat
T-15	generateCSVString should format headers and rows correctly	Przekazanie nagłówków Date, Value oraz dwóch wierszy z datami i kursami.	Zwrócony tekst CSV zawiera nagłówki i wiersze w cudzysłowach, rozdzielone przecinkami i znakami nowej linii.
T-16	generateCSVString should escape double quotes within cells	Przekazanie komórki tekstowej zawierającej cudzysłowy: This is a "quoted" comment.	Cudzysłowy wewnętrzne zostają zdublowane: "This is a ""quoted"" comment".

6.7. End-to-End User Flow Simulation

ID	Test	Procedura	Oczekiwany rezultat
T-17	Flow: User selects currency, runs analysis, and views statistical results	Utworzenie uproszczonego DOM, zamockowanie odpowiedzi API z dwoma kursami i wywołanie ui.run().	results.hidden = false, status = 'Success.', liczba elementów .stat-card > 0, exportBtn.disabled = false.
T-18	Flow: User toggles between Chart and Table views	Pobranie zakładek chart i table, sprawdzenie stanu początkowego, wywołanie ui.switchView(tableTab).	Po przełączeniu chartWrap.hidden = true, a tableWrap.hidden = false.

7. Dane testowe i mockowanie

Dane testowe są definiowane bezpośrednio w pliku app.test.js. Odpowiedzi API są symulowane przez mocki fetch.mockResolvedValue() lub fetch.mockResolvedValueOnce(). Dzięki temu testy są deterministyczne i nie zależą od dostępności sieci ani od aktualnego stanu API NBP.

```
global.fetch = jest.fn();
```

```
fetch.mockResolvedValue({
  ok: true,
  json: () => Promise.resolve({
    rates: [
      { effectiveDate: '2023-01-01', mid: 4.0 },
      { effectiveDate: '2023-01-02', mid: 5.0 }
    ]
  })
});
```

W testach przepływu użytkownika tworzony jest minimalny DOM wymagany przez UIController. Nie jest ładowany pełny plik index.html, dlatego test ten należy traktować jako symulację przepływu, a nie pełny test interfejsu w przeglądarce.

8. Wniosek końcowy

Procedura testowa oparta na app.test.js jest wykonywana automatycznie w GitHub Actions po push lub pull request na gałęzi main, release i develop. Sprawdza przede wszystkim poprawność działania logiki JavaScriptowej aplikacji oraz integrację głównych klas odpowiedzialnych za pobieranie danych, analizę statystyczną, analizę sesji, rozkład zmian i eksport CSV. Ostatnia grupa testów częściowo symuluje przepływ użytkownika w JSDOM, ale nie stanowi pełnego testu end-to-end.

Na podstawie zawartości `app.test.js` można stwierdzić, że zestaw testów dobrze pokrywa funkcje obliczeniowe oraz komunikację między modułami aplikacji. Do pełnej akceptacji działania aplikacji jako produktu webowego należy jednak uzupełnić procedurę o testy manualne lub automatyczne E2E wykonywane w rzeczywistej przeglądarce.